

QA Test Guide — Step by Step

Printable walkthrough generated from [docs/QA_TEST_PLAN.md](#). Each case shows its Gherkin scenario and the plain-English steps. Record results on the companion [QA Run Log](#) sheet. Generated 2026-06-26.

End-to-end manual test plan covering the **entire** functionality of the app across all three clients: **Web** (Blazor WASM), **Desktop** (MAUI / Windows), and **Android** (MAUI). Each case is written twice — a **Gherkin** scenario (Given/When/Then, matching this project's convention so it can later seed the automated E2E . Tests Playwright suite) and a ****plain-English walkthrough**** a manual tester can follow step by step.

Brand in examples is "Perezosoft"; substitute your app's brand if rebranded.

How to use this document

- **Run the Smoke suite (§4) first.** It's the ~15-minute critical path. If any smoke case fails, stop and report — deeper suites will likely cascade.
- **Each case has an ID** (e.g. QA-AUTH-03). Record the result against the ID using the **sign-off sheet (§14)**: Pass / Fail / Blocked / N-A, plus tester, build/commit, date, notes.
- **Priority:** ● **Smoke** Smoke (critical path) · ● **Core** Core (run every regression) · ● **Edge** Edge (run on full regression or when the area changed).
- **Both formats describe the same test.** Read whichever suits you; the Gherkin is the source of truth for automation.
- **"App" = whichever client the suite header names.** Most behavior is identical across clients (same API, same shared RCL UI); the per-client suites (§11–12) only cover what genuinely differs — the auth transport and session persistence.

1. Environment & prerequisites

1.1 Backing services + API (host machine)

```
docker compose up -d # Postgres + Mailpit
dotnet run --project src/Api --launch-profile https # binds https:7160 (web/desktop) AND http:5238 (android)
```

- API health/liveness check: `curl -k https://localhost:7160/health → 200` (Healthy);
`curl -k https://localhost:7160/health/ready → 200` when the database is reachable (503 if not). *(The older `curl -k -X POST https://localhost:7160/api/auth/refresh → 401` reachability check still works.)*
- **Mailpit UI:** <http://localhost:8025> — this is the dev mail trap. Every magic link, OTP code, and invitation email lands here. Keep it open in a tab throughout testing.

■ Email only reaches Mailpit if SMTP points at it. If your repo-root .env has the Email__Smtp__* lines set to a real provider (e.g. Brevo), the API sends auth emails there and Mailpit stays empty — every email-based case below will appear to "fail." For QA, route mail to Mailpit by either:

- commenting out the Email__Smtp__* lines in .env (unset → defaults to Mailpit localhost:1025), or
- leaving .env untouched and overriding on the command line (command-line config beats .env):

```
```bash
dotnet run --project src/Api --launch-profile https -- \
--Email:Smtp:Host=localhost --Email:Smtp:Port=1025 --Email:Smtp:Username= --Email:Smtp:Password=
```
```

Verify by triggering one OTP (QA-SMK-01) and confirming it appears in Mailpit before running the suite.

■ Email delivery is now asynchronous. Requesting a code/link/invite enqueues the email and a background dispatcher (the outbox) sends it — so it appears in Mailpit **a few seconds later, not instantly. Wait briefly before assuming failure. The send request now always returns success** (reliability moved to the background): if an email never arrives while SMTP points at Mailpit, the message is retrying or dead-lettered in the OutboxMessages table — it is no longer surfaced as a request error.

- Web app: dotnet run --project src/Web --launch-profile https → <https://localhost:7008>.

■ Always use the https profile for both web and API. Chrome treats http://localhost and https://localhost as different sites, so the refresh cookie is dropped over http and sign-in silently fails to persist. (See docs/DECISIONS.md / the schemeful-same-site note.)

■ The passwordless endpoints are rate-limited (default 5 requests/minute per IP on /otp/send, /magic-link/send, and /otp/verify). If you fire many code/link requests or verify attempts in quick succession you may get HTTP 429 — that's the abuse guard working (QA-AUTH-11), not a bug. Pace requests, or wait ~1 minute for the window to reset.

1.2 Test accounts & data

| Need | What to prepare |
|--------------------------------|--|
| Two real Google accounts | e.g. qa.owner@gmail.com, qa.member@gmail.com — for OAuth + linking + multi-user household flows. |
| One Microsoft personal account | for the Microsoft OAuth path (provider pinned to the *consumers* tenant). |
| Throwaway email addresses | any address works for magic-link / OTP — mail is trapped by Mailpit, so the address need not be real. Use distinct ones per test to keep inboxes clean. |
| Two browser contexts | a normal window and an incognito/second-profile window. The household invite flow needs two *different* signed-in users at once; incognito gives you an isolated session + cookie jar. |

Database reset between full runs (optional but recommended): to retest "new user" onboarding cleanly, you need users that don't yet exist. Either use fresh email addresses each run, or reset the dev DB: docker compose down -v && docker compose up -d, then re-apply migrations by starting the API. A volume wipe destroys all test data — only do it on the dev environment.

1.3 Desktop (MAUI Windows) — additional setup

- Run the app from Visual Studio (Windows Machine target) or dotnet build src/Maui -t:Run -f net10.0-windows....
- API must be running on https://localhost:7160 (the desktop client's base URL).
- OAuth uses a loopback browser flow — your default system browser will open a tab during OAuth.

1.4 Android (MAUI) — additional setup

Follow docs/MOBILE_TESTING.md. The essential bits:

- Emulator (AVD) or USB device running.
- **adb reverse tcp:5238 tcp:5238 — re-run every time the device/emulator restarts** (it does not persist). Verify with adb reverse --list.

- API started with the **https** profile (binds the cleartext : 5238 leg the device uses).
- Provider redirect URIs registered: <http://localhost:5238/signin-google> and <http://localhost:5238/signin-microsoft>.

2. Scope

In scope: authentication (all methods, all clients), new-user onboarding, household/tenant management, invitations & joining, account settings (linked providers), localization (EN/ES), transactional emails, and cross-cutting security (tenant isolation, auth guards, token lifecycle).

****Out of scope (per docs/PROJECT_BRIEF.md OUT list & current state):**** SMS OTP, OAuth providers beyond Google/Microsoft, FR/DE/PT languages (scaffolded but not translated — see docs/LOCALIZATION.md), and any app-specific domain features not yet built on this template.

Platform services with no client UI (API/operational-level, not manually testable through the app yet): the billing API (/api/billing/*), the append-only audit log, OpenTelemetry telemetry, the health endpoints, and the background outbox/inbox/scheduled-jobs. These are **covered by automated tests** (`tests/Api.Tests`); E2E is pending. Health has a smoke check (QA-SMK-07); manual cases for the rest will be added when client UI exists.

3. Test data conventions

- **Owner user** = the account you sign in with first; it auto-creates and owns a household.
- **Member user** = a second account invited into the owner's household.
- Household auto-named on creation; rename it to something recognizable (e.g. "QA House") early so you can spot it in the header tenant badge.

4. Smoke suite ● Smoke (run first — ~15 min)

The critical path: can a user get in by each method, reach the app, and get out.

QA-SMK-01 — Web: OTP sign-in happy path ● Smoke (Web)

Gherkin

```
Given I am an anonymous user on the /login page of the web app
When I enter my email and request a 6-digit code
And I retrieve the code from Mailpit and submit it
Then I am signed in and land on the home page
And the header shows my household name and my display name
```

Walkthrough

- 1 Open <https://localhost:7008> → you're redirected to `/login`.
- 2 In the email field enter `qa-smoke@example.com`; click **Email me a 6-digit code**.
- 3 **Expected:** the form switches to a code-entry view ("Enter the code we sent to...").
- 4 Open Mailpit (<http://localhost:8025>); open the newest mail; copy the 6-digit code.
- 5 Enter the code; click **Verify code**.
- 6 **Expected:** you land on the home page; the top header shows a tenant badge (household name) and your display name, plus **Household**, **Settings**, **Sign out** buttons.

QA-SMK-02 — Web: Google OAuth sign-in ● Smoke (Web)

Gherkin

```
Given I am on the /login page
When I click "Continue with Google" and complete Google consent
Then I am returned to the app, signed in, on the home page
```

Walkthrough

- 1 From /login, click **Continue with Google**.
- 2 **Expected:** full-page redirect to Google's consent screen.
- 3 Choose qa.owner@gmail.com, approve.
- 4 **Expected:** redirected back into the app, signed in, on the home page with the header chrome.

QA-SMK-03 — Web: Sign out ● Smoke (Web)

Gherkin

```
Given I am signed in to the web app
When I click "Sign out"
Then I am returned to the /login page
And navigating to /settings redirects me back to /login
```

Walkthrough

- 1 While signed in, click **Sign out** in the header.
- 2 **Expected:** you land on /login.
- 3 In the address bar go to /settings.
- 4 **Expected:** you're bounced back to /login (no access without a session).

QA-SMK-04 — Web: Session persists across reload ("remember me") ● Smoke (Web)

Gherkin

```
Given I am signed in to the web app
When I reload the page (or reopen the tab)
Then I am still signed in without re-authenticating
```

Walkthrough

- 1 Signed in, press F5 / reload.
- 2 **Expected:** brief load, then the app shell — still signed in, no trip to /login. (A silent refresh exchanges the refresh cookie for a new access token on load.)

QA-SMK-05 — Desktop: OTP sign-in ● Smoke (Desktop) — see QA-DSK-01

QA-SMK-06 — Android: OTP sign-in ● Smoke (Android) — see QA-AND-01

QA-SMK-07 — API health & readiness ● Smoke (Platform)

Gherkin

```
Given the API is running
When I GET /health and /health/ready
Then /health returns 200 "Healthy"
And /health/ready returns 200 when the database is reachable, 503 when it is not
```

Walkthrough

- 1 `curl -k https://localhost:7160/health → 200`, body Healthy (liveness — process up).
- 2 `curl -k https://localhost:7160/health/ready → 200` (readiness — Postgres reachable).
- 3 **(Optional)** stop the DB (`docker compose stop db`) and re-run step 2 → **503**; then `docker compose start db` and confirm it returns to **200**.
- 1 **Expected:** liveness is 200 whenever the process runs; readiness tracks DB reachability. Responses are **status-only** (no connection details leaked).

5. Web — Authentication ● Core

QA-AUTH-01 — Magic-link sign-in happy path ● Core (Web)

Gherkin

Given I am an anonymous user on /login
When I enter my email and request a magic link
And I open the emailed link from Mailpit
Then I am signed in and landed in the app

Walkthrough

- 1 On /login enter qa-magic@example.com; click **Send me a magic link**.
- 2 **Expected:** a success panel — "Check your inbox... we sent a link to qa-magic@example.com" — with a **Use a different email** link.
- 1 In Mailpit open the newest mail; click the sign-in link (or copy it into the same browser).
- 2 **Expected:** the link resolves and you end up signed in, in the app.

QA-AUTH-02 — Microsoft OAuth sign-in ● Core (Web)

Gherkin

Given I am on /login
When I click "Continue with Microsoft" and complete consent
Then I am returned signed in

Account resolution (read before testing): signing in with a provider whose email matches an existing account links the provider to that account rather than creating a duplicate — but only when the email is **verified**. Google asserts this; Microsoft on the ****consumers**** (personal MSA) tenant is trusted to have a verified email even though it omits the claim (IPProviderEmailTrust). For **work/school** tenants (organizations/common/a tenant GUID) and any other provider, a same-email auto-link is **refused** (fail-closed takeover guard, audit MITI-3) — the user signs in with their original method and links the provider from **Settings** instead (QA-SET-02). To test the plain first-time path below, use a Microsoft account whose email has **no** prior account.

Walkthrough

- 1 Click **Continue with Microsoft**; sign in with a **personal** Microsoft account.
- 2 **Expected:** returned to the app signed in. A brand-new email creates a new account; a personal Microsoft account whose email already exists auto-links to that account (consumers tenant). (If you see a tenant/reply-URL error, the provider registration is the cause — out of app scope, note it.)

QA-AUTH-03 — OTP wrong code is rejected ● Core (Web)

Gherkin

Given I requested an OTP code for my email
When I submit an incorrect 6-digit code
Then I see an "incorrect code" error and remain on the code-entry screen

Walkthrough

- 1 Request an OTP (as QA-SMK-01) but **do not** use the real code.
- 2 Enter 000000; click **Verify code**.
- 3 **Expected:** inline error ("code is incorrect / verification failed"); you stay on the code-entry view and can retry. You are **not** signed in.

QA-AUTH-04 — OTP cumulative lockout & code expiry ● Edge (Web)

Gherkin

Given I requested an OTP code
When I submit wrong codes up to the cumulative limit (default 5 failures per email within a 15-min window)
Then the email is locked – further attempts, INCLUDING a freshly requested code, are rejected until the window elapses
And separately, an unused code is rejected once it passes its lifespan (default 10 minutes)

Walkthrough

- 1 Request an OTP for an email; enter a **wrong** 6-digit code until you hit the cumulative limit

(default 5, `Auth:Otp:MaxAttempts`, counted **per email across the `Auth:Otp:LockoutWindowMinutes` = 15-min window**, not per code).

- 1 **Expected:** after the limit, a "too many attempts" lockout error.
- 2 Now request a **fresh** code and submit it — even the **correct** one.
- 3 **Expected:** still rejected — requesting a new code does **NOT** reset the budget (this is the brute-force defense; a resend can't hand the attacker another N guesses). The lock clears after the 15-min window.
- 1 **Expiry sub-case:** separately, request a code and wait past `Auth:Otp:CodeLifespanMinutes` (default 10) — the stale code is rejected.

Note (enumeration): OTP verify returns the **same** generic "invalid code" error whether the code was wrong or there is no active code — it never reveals whether an address has an outstanding OTP. *(Long-wait cases — lower the config to test fast. Mind the per-IP rate limit, QA-AUTH-11: 5 wrong verifies in a minute also trips the 429 throttle.)*

QA-AUTH-11 — Passwordless endpoints are rate-limited ● Core (Web)

Gherkin

```
Given I rapidly request codes/links (or verify attempts) for the same client
When I exceed the per-IP limit (default 5 per minute)
Then further requests are rejected with HTTP 429 until the window resets
```

Walkthrough

- 1 From /login, request an OTP (or magic link) repeatedly in quick succession — more than 5 within a minute.
- 1 **Expected:** after the limit the request is throttled (**HTTP 429**, surfaced as a "try again shortly" / send-failed state). The same throttle applies to **OTP verify**.
- 1 Wait ~1 minute; requests succeed again.

Protects against email-bombing and OTP brute-forcing. Expected behavior, not a defect — see the §1.1 pacing note.

QA-AUTH-05 — Magic link is single-use ● Edge (Web)

Gherkin

```
Given I signed in by clicking a magic link
When I click the same link a second time
Then it is no longer valid
```

Walkthrough

- 1 Complete QA-AUTH-01. Then revisit the *same* link from Mailpit.
- 2 **Expected:** it does not grant a second session — you're sent to /login with an invalid-link indication (or simply not signed in). The token is consumed on first use.

QA-AUTH-06 — Magic link expiry ● Edge (Web)

Walkthrough: request a link, wait past `Auth:MagicLink:TokenLifespanMinutes` (default 15), then open it.

Expected: rejected as expired; user lands on /login. *(Lower the config to test fast.)*

QA-AUTH-07 — User-enumeration protection ● Core (Web)

Gherkin

```
Given an email address that has no account
When I request a magic link or OTP for it
Then the UI shows the same success/"check your inbox" response as for a real account
```

Walkthrough

- 1 Request a magic link **and** an OTP for a never-before-used address.
- 2 **Expected:** identical success messaging to a known address — the app never reveals whether an

account exists. (Mailpit will still show whatever the system chose to send.)

QA-AUTH-08 — Login error banners ● Edge (Web)

Gherkin

```
Given the login page is loaded with an error query parameter
Then a human-readable error banner is shown
```

Walkthrough — visit each URL and confirm a red banner with sensible copy:

- `/login?error=external_failed` → external sign-in failed message.
- `/login?error=email_unverified` → email-unverified message.
- `/login?error=invalid_link` → invalid/expired link message.
- `/login?error=somethingelse` → generic "something went wrong" fallback.

QA-AUTH-09 — Email-format validation ● Edge (Web)

Walkthrough: on `/login`, click a send button with an empty or malformed email (e.g. `abc`). **Expected:** inline "enter a valid email" validation; no request sent.

QA-AUTH-10 — Magic link & OTP available on web; OAuth always ● Edge (Web)

Walkthrough: confirm the web login page shows **both** "Send magic link" and "Email me a code", plus Google/Microsoft buttons. (Native clients hide magic link — covered in §11–12.)

6. Web — Onboarding & new tenant ● Core

QA-ONB-01 — First-ever sign-in auto-creates a household, user is owner ● Smoke (Web)

Gherkin

```
Given an email/account that has never signed in before
When I authenticate for the first time (any method)
Then a new household is created and I am its owner
And the header shows that household and my name
```

Walkthrough

- 1 Sign in with a brand-new account/email (fresh address or post-DB-reset).
- 2 **Expected:** you reach the app immediately (no "create household" prompt — provisioning is automatic). Open **Household**: you are listed with the **Owner** badge and are the only member.

QA-ONB-02 — Returning user keeps their household ● Core (Web)

Walkthrough: sign out and back in with the same account. **Expected:** same household, same role, data intact.

7. Web — Household management ● Core

Owner-only controls: rename, invite, remove member, transfer ownership, dissolve. Members see a read-only name and a **Leave** button.

QA-HH-01 — Owner renames the household ● Core (Web)

Gherkin

```
Given I am the household owner on /household
When I change the name and click Rename
Then the new name is saved and reflected in the header tenant badge
```

Walkthrough

- 1 **Household** → edit the name field (e.g. "QA House") → **Rename**.
- 2 **Expected:** success banner; the header tenant badge updates to the new name (a silent refresh

updates the claim).

QA-HH-02 — Member sees read-only household, no owner controls ● Core (Web)

Precondition: signed in as a *member* (use QA-INV flow to create one). **Walkthrough:** open **Household**. **Expected:** household name shown as read-only heading; no rename/invite/transfer controls; a **Leave** button is present.

QA-HH-03 — Owner removes a member ● Core (Web)

Gherkin

```
Given I am the owner and another member exists
When I click Remove next to that member and confirm
Then they are removed from the household member list
```

Walkthrough

- 1 As owner with a member present, click **Remove** by the member's row.
- 2 **Expected:** a confirm dialog ("Remove <name>?"); on confirm, success banner and the member disappears from the list. (The removed user is re-homed to a fresh solo household — verify by signing in as them: they now own an empty household — see QA-HH-08.)

QA-HH-04 — Owner cannot remove themselves ● Edge (Web)

Walkthrough: as owner, confirm there is **no Remove button on your own row**. (Owner departs via transfer or dissolve, not removal.)

QA-HH-05 — Transfer ownership ● Core (Web)

Gherkin

```
Given I am the owner and at least one other member exists
When I select that member and click Transfer
Then they become owner and I become a regular member
```

Walkthrough

- 1 As owner with ≥ 1 other member, in **Transfer ownership** pick the member → **Transfer**.
- 2 **Expected:** success banner. The chosen member now shows the **Owner** badge; your row shows **Member**. The owner-only controls (invite/transfer/dissolve) are no longer available to you, and a **Leave** button now is.

QA-HH-06 — Owner with other members cannot directly leave/dissolve ● Edge (Web)

Walkthrough: as owner **with** other members present, confirm the bottom card shows the **Transfer ownership** control (with "you must transfer before leaving" note) — **not** a leave/dissolve button. The single-owner invariant is enforced in the UI.

QA-HH-07 — Sole owner dissolves the household ● Core (Web)

Gherkin

```
Given I am the only member and owner of my household
When I click "Leave and delete household" and confirm
Then the household is dissolved and I am re-homed to a fresh solo household
```

Walkthrough

- 1 As sole owner (no other members), bottom card → **Leave & delete household**.
- 2 **Expected:** a confirm dialog warning the household will be deleted; on confirm, the app reloads and you land signed in with a **new empty household you own** (you're never left tenant-less).

QA-HH-08 — Member leaves the household ● Core (Web)

Gherkin

```
Given I am a non-owner member
When I click Leave and confirm
Then I leave the household and am re-homed to a fresh solo household I own
```


Walkthrough

- 1 As a member, **Household** → **Leave** → confirm.
- 2 **Expected:** app reloads; you now own a brand-new empty household. The household you left still exists for its remaining members (verify as the owner: the leaver is gone from the member list).

8. Web — Invitations & joining ● Core

QA-INV-01 — Owner invites by email; token revealed + email sent ● Smoke (Web)

Gherkin

```
Given I am the household owner
When I invite an email address
Then an invitation appears in the pending list with an expiry date
And a one-time join token is revealed in the UI
And an invitation email with a /join link is delivered to Mailpit
```

Walkthrough

- 1 **Household** → Invitations → enter `qa.member@gmail.com` → **Invite**.
- 2 **Expected:** a green panel reveals the raw token + a **Copy** button and notes a join link was emailed; the address shows under **Pending** with an expiry date.
- 1 Check Mailpit: an invitation email addressed to that user, containing a `/join?token=...` link.

QA-INV-02 — Invitee accepts and joins (two-user flow) ● Smoke (Web)

Gherkin

```
Given an invitation exists for a second user
When that user opens the /join link, signs in, and accepts
Then they become a member of the inviter's household
```

Walkthrough

- 1 Copy the `/join?token=...` link from QA-INV-01.
- 2 In an **incognito window**, open the link.
- 3 **Expected:** "You've been invited... sign in to accept." Click **Sign in to accept**; complete any sign-in method as `qa.member@gmail.com`.
- 1 **Expected:** after sign-in you're returned to the join flow automatically, it processes, and you see "You're in" with a **Go to household** button.
- 1 Click it → **Household** shows you as a **Member** of "QA House".
- 2 Back in the owner window, reload **Household**: the new member appears and the pending invite is gone.

QA-INV-03 — Already-authenticated user accepts directly ● Core (Web)

Walkthrough: while already signed in as a *different* fresh user, open a valid `/join?token=...` **Expected:** it accepts immediately (no sign-in step) and shows success. *(Note: a user already in a household who accepts another invite moves to the new household — verify their old membership is replaced, honoring the one-tenant invariant.)*

QA-INV-04 — Join link with missing token ● Edge (Web)

Walkthrough: open `/join` with no `?token=`. **Expected:** "invalid invitation / missing token" state, no crash.

QA-INV-05 — Join with invalid/expired/used token ● Core (Web)

Gherkin

```
Given an invitation token that is invalid, already used, or expired
When an authenticated user opens its /join link
Then they see an error state, not a successful join
```

Walkthrough: while signed in, open `/join?token=garbage123` (or reuse a token already accepted in QA-INV-02).

Expected: the error state with a **Back to household** link; no membership change.

QA-INV-06 — Invite an existing member is rejected ● Core (Web)

Gherkin

```
Given a user is already a member of my household
When I invite their email again
Then I get an "already a member" error and no duplicate invite is created
```

Walkthrough: as owner, invite the email of someone already in the household. **Expected:** a red "already a member" status (HTTP 409); nothing added to pending.

QA-INV-07 — Regenerate a pending invitation ● Core (Web)

Gherkin

```
Given a pending invitation exists
When I click Regenerate
Then a new token is issued (revealed) and the old token no longer works
```

Walkthrough

- 1 On a pending invite, click **Regenerate**.
- 2 **Expected:** a new token is revealed. The **previous** token's `/join` link now fails (QA-INV-05), the new one works.

QA-INV-08 — Revoke a pending invitation ● Core (Web)

Gherkin

```
Given a pending invitation exists
When I click Revoke
Then it is removed from pending and its token no longer works
```

Walkthrough: click **Revoke** on a pending invite. **Expected:** "invitation revoked" status; it leaves the pending list; opening its old link gives the error state.

QA-INV-09 — Copy token button ● Edge (Web)

Walkthrough: click **Copy** on a revealed token; paste elsewhere. **Expected:** the token is on the clipboard. (If the browser blocks clipboard access, the token is still visible to copy manually — no error shown.)

9. Web — Settings / linked accounts ● Core

QA-SET-01 — View linked providers ● Core (Web)

Gherkin

```
Given I signed up with Google
When I open /settings
Then Google shows as "Connected" and Microsoft shows a "Link" button
```

Walkthrough: sign in with Google, open **Settings**. **Expected:** a row per provider; the one you used shows a **Connected** badge + **Unlink**; the other shows **Link**.

QA-SET-02 — Link a second provider ● Core (Web)

Gherkin

```
Given I am signed in and Microsoft is not linked
When I click Link on Microsoft and complete consent
Then Microsoft becomes Connected on my account (no new account is created)
```

Walkthrough

- 1 **Settings** → **Link** on Microsoft → complete consent with a Microsoft account **whose email

isn't already used by another app user**.

- 1 **Expected:** returned to Settings with a success banner ("Microsoft linked"); Microsoft now shows **Connected**. You can subsequently sign in with either provider into the *same* account.

QA-SET-03 — Linking a provider already used by another account is rejected ● Core (Web) Gherkin

```
Given a provider identity is already linked to a different user
When I try to link it to my account
Then I get an "already in use" error and it is not linked
```

Walkthrough: try to **Link** a Google/Microsoft identity that another app user already owns. **Expected:** Settings shows an "already in use" banner (?link_error=in_use); no link made. *(Expired link-token path shows ?link_error=expired — exercise if you can stall the flow past the token lifetime.)*

QA-SET-04 — Unlink a provider ● Core (Web) Gherkin

```
Given two providers are linked to my account
When I click Unlink on one and confirm the dialog
Then it returns to a "Link" state
```

Walkthrough: with two providers connected, click **Unlink** on one. **Expected:** a confirm dialog ("Unlink this sign-in method?"); on **confirm** the row reverts to **Link**. **Cancelling** the dialog leaves it **Connected** — no change. *(The confirm fails closed: if the browser dialog can't run, the unlink is cancelled, never silently performed. The same guarded confirm protects remove-member / leave / dissolve in §7.)*

QA-SET-05 — Unlinking your only provider never locks you out ● Edge (Web) Gherkin

```
Given Google is my only linked provider
When I unlink it
Then I can still sign in by email (magic link / OTP)
```

Walkthrough: unlink your sole provider, sign out, and sign back in with **OTP/magic link** using the same email. **Expected:** you reach the **same** account/household. (Email sign-in is always available by design, so provider removal can't lock you out.)

QA-SET-06 — Settings requires auth ● Edge (Web)

Walkthrough: signed out, navigate to /settings. **Expected:** redirect to /login.

10. Web — Localization (i18n) ● Core

QA-I18N-01 — Switch language on the login page ● Core (Web) Gherkin

```
Given I am on /login in English
When I choose Español in the language switcher
Then the page text renders in Spanish
```

Walkthrough: on /login, use the language switcher (bottom of the card) → **Español**. **Expected:** titles, button labels, and prompts switch to Spanish; the choice persists on reload.

QA-I18N-02 — Language persists per user across sessions ● Core (Web) Gherkin

```
Given I am signed in and set my language to Spanish
When I sign out and sign back in (even in a fresh browser)
Then the app loads in Spanish
```

Walkthrough

- 1 Signed in, switch to **Español**. (This saves the locale to your user record via ``PUT /api/auth/locale`` and into the JWT.)
- 1 Sign out; sign back in.
- 2 **Expected:** app comes up in Spanish — the preference followed the **user**, not just the browser.

QA-I18N-03 — In-app UI is fully translated (no English leaks) ● Edge (Web)

Walkthrough: in Spanish, walk Home → Household → Settings → invite flow. **Expected:** all visible labels/buttons/validation/status messages are Spanish; flag any English string that leaks.

QA-I18N-04 — Email language matches the requester's UI language ● Core (Web)

Gherkin

```
Given my UI language is Spanish
When I trigger an OTP / magic link / invitation email
Then the email arrives in Spanish
```

Walkthrough

- 1 Set UI to **Español**. Trigger an OTP (and a magic link, and send an invite).
- 2 In Mailpit, open each. **Expected:** subject + body in Spanish. Repeat in English to confirm both.

11. Emails (Mailpit) — branding & content ● Core

Delivery is asynchronous (the outbox dispatcher) — emails land in Mailpit a few seconds after the triggering action, and the send request succeeds regardless (see §1.1). Wait briefly before opening Mailpit; a missing email is a delayed/retrying/dead-lettered outbox message, not a request failure.

QA-MAIL-01 — OTP email is branded & correct ● Core

Gherkin

```
Given I request an OTP code
When I open the email in Mailpit
Then it shows the brand logo, brand colours, the 6-digit code, and the tagline
```

Walkthrough: trigger an OTP; open it in Mailpit. **Expected:** the logo image renders (CID-embedded, not a broken image), brand-green styling, a clearly displayed code, footer wordmark + tagline.

QA-MAIL-02 — Magic-link email ● Core

Walkthrough: trigger a magic link; open in Mailpit. **Expected:** branded layout; a working sign-in button/link; sensible subject.

QA-MAIL-03 — Invitation email ● Core

Walkthrough: send an invite; open in Mailpit. **Expected:** branded layout; the recipient's address; a `/join?token=...` link that works (QA-INV-02).

QA-MAIL-04 — Logo renders in a real client ● Edge

Walkthrough (optional, deliverability): forward/inspect one email in a real Gmail/Outlook client. **Expected:** the CID logo still renders. *(Note: from a non-verified domain, real delivery may land in spam — a domain/DKIM concern, not an app bug.)*

12. Desktop — MAUI / Windows ● Core

The desktop client reuses the same UI; only the **auth transport** (loopback browser flow, body token, OS secure storage) differs. Magic link is intentionally **absent** on native.

QA-DSK-01 — OTP sign-in ● Smoke (Desktop)

Gherkin

Given the desktop app is on the login screen
When I request an OTP and enter the code from Mailpit
Then I am signed in within the app (no browser)

Walkthrough

- 1 Launch the desktop app → login screen. **Expected:** Google/Microsoft buttons + email field with **Email me a code; no magic-link button** (native hides it).
- 1 Enter an email → request code → get it from Mailpit → enter it.
- 2 **Expected:** signed in, app shell shown, household + name in header.

QA-DSK-02 — Google OAuth via loopback ● Core (Desktop)

Gherkin

Given the desktop login screen
When I click Continue with Google and complete consent in the system browser
Then the browser shows a "you can close this" page and the app completes sign-in

Walkthrough

- 1 Click **Continue with Google**. **Expected:** your **system browser** opens to Google consent.
- 2 Approve. **Expected:** the browser tab shows a "you can close this tab" page; ****focus returns to the app, now signed in. (Repeat for Microsoft**.)**

QA-DSK-03 — "Remember me" across app restart ● Core (Desktop)

Gherkin

Given I am signed in to the desktop app
When I fully close and reopen it
Then I am still signed in

Walkthrough: close the app completely; relaunch. **Expected:** lands signed in (refresh token held in Windows secure storage / DPAPI, silently exchanged on startup).

QA-DSK-04 — Household & Settings load (authorized API calls) ● Core (Desktop)

Gherkin

Given I am signed in to the desktop app
When I open Household and Settings
Then both load their data without an auth error

Walkthrough: open **Household** (members load) and **Settings** (linked providers load). **Expected:** both populate — no spinner-forever, no "couldn't load" error. *(This is the Bearer-token-handler path that previously failed on native; it must work.)*

QA-DSK-05 — Link a provider from Settings ● Edge (Desktop)

Walkthrough: **Settings** → **Link Microsoft** → system browser loopback flow → returns to app. **Expected:** Microsoft shows **Connected**; the app shell was never navigated away.

QA-DSK-06 — Sign out ● Edge (Desktop)

Walkthrough: **Sign out**. **Expected:** back to the login screen; reopening the app does **not** auto-sign-in (secure storage cleared).

QA-DSK-07 — Core household flows work on desktop ● Edge (Desktop)

Walkthrough: spot-check rename, invite (token revealed), and leave on desktop. **Expected:** same behavior as web (§7–8) — the UI is shared.

13. Android — MAUI ● Core

Prereq every run: `**adb reverse tcp:5238 tcp:5238**` + API on the https profile. See docs/MOBILE_TESTING.md.

QA-AND-01 — OTP sign-in ● Smoke (Android)

Gherkin

```
Given the Android app is on the login screen with adb reverse set
When I request an OTP and enter the code from Mailpit
Then I am signed in
```

Walkthrough

- 1 Confirm `adb reverse --list` shows `tcp:5238`. Launch the app.
- 2 Login screen: email field + **Email me a 6-digit code**; Google/Microsoft buttons; `**no magic link**`.
- 1 Enter an email → request code → read it in Mailpit (host) → enter it.
- 2 **Expected:** signed in, app shell shown.

QA-AND-02 — Google OAuth via custom scheme ● Core (Android)

Gherkin

```
Given the Android login screen
When I tap Continue with Google and complete consent
Then the in-app browser returns to the app via the perezosoftware:// scheme, signed in
```

Walkthrough

- 1 Tap **Continue with Google**. **Expected:** a browser tab opens to Google consent.
- 2 Approve. **Expected:** the tab returns control to the app (`perezosoftware://auth` intent), now signed in. (Repeat **Microsoft** if its `:5238` redirect is registered.)

QA-AND-03 — "Remember me" across app restart ● Core (Android)

Walkthrough: swipe-close the app; reopen. **Expected:** still signed in (refresh token in the Android Keystore). *(If `adb reverse` was lost on a device reboot, re-run it first — a startup failure after reboot is an environment issue, not an app bug.)*

QA-AND-04 — Household & Settings load ● Core (Android)

Walkthrough: open **Household** and **Settings**. **Expected:** both load their data (the native Bearer path works), same as desktop QA-DSK-04.

QA-AND-05 — Navigation drawer / header is reachable ● Edge (Android)

Gherkin

```
Given I am signed in on Android
When I open the navigation (hamburger)
Then the menu is tappable and not hidden under the status bar
```

Walkthrough: tap the hamburger; use **Household/Settings/Sign out**. **Expected:** the header sits below the status bar (safe-area padding) and every item is tappable.

QA-AND-06 — Core flows on Android ● Edge (Android)

Walkthrough: spot-check language switch, invite (token revealed), and leave. **Expected:** parity with web.

14. Cross-cutting security ● Core

QA-SEC-01 — Tenant isolation ● Smoke

Gherkin

```
Given two households owned by two different users
When user A is signed in
Then A can only ever see A's household, members, and invitations – never B's
```

Walkthrough

- 1 Create household A (owner A) and a separate household B (owner B, different account/incognito).
 - 2 As A, open **Household**. **Expected:** only A's members/invites. Confirm B's data never appears.
- *(Deeper API-level probing — e.g. requesting another tenant's resource id — belongs in automated `Api.Tests`; this manual case verifies the UI never cross-contaminates.)*

QA-SEC-02 — Protected pages require auth ● Core

Walkthrough: signed out, directly visit `/household`, `/settings`. **Expected:** each redirects to `/login`.

QA-SEC-03 — Session is gone after sign-out ● Core

Gherkin

```
Given I sign out
When I press the browser Back button or reload a protected page
Then I am not able to access it – I am sent to /login
```

Walkthrough: sign out, press **Back** to a protected page / reload it. **Expected:** bounced to `/login`; no stale authenticated view.

QA-SEC-04 — Native open-redirect guard ● Edge (Desktop/Android)

Context/Expected: the native OAuth flow only honors loopback `http` callbacks or the configured `perezosoftware://` scheme; arbitrary redirect targets are rejected. This is unit-tested (`NativeRedirectPolicyTests`); no manual action needed unless probing the API directly — record as **covered by automated tests**.

QA-SEC-05 — Server-side hardening (automated) ● Edge

Context/Expected: several invariants are enforced at the API/data layer and verified by `tests/Api.Tests`, not by manual UI steps — record as **covered by automated tests** unless probing the API directly:

- **Tenant write-stamping** — a new tenant-scoped row is stamped with the caller's tenant and a foreign-tenant write is rejected (`TenantStampingInterceptorTests`), so reads *and* writes are tenant-isolated.
- **Refresh-token reuse detection** — replaying an already-rotated refresh token revokes all the user's sessions (`RefreshTokenServiceTests`). Manually observable only by capturing and replaying a refresh cookie/token; out of scope for routine QA.
- **Unverified-email takeover guard** fails closed (`ClaimsExtractorTests / UserServiceTests`).
- **Legacy refresh-cookie self-heal** — a stale `Path=/` refresh cookie left by an older build can shadow the live `Path=/api/auth` cookie and wedge sign-in into a `/refresh 401` → "Authentication Failed" loop (seen in Firefox, due to cookie send-order). Setting the refresh cookie now also emits an expiry for the orphan, so the next successful sign-in / refresh sweeps it automatically — no manual cookie-clearing, and upgraded deployments self-heal (`CookieServiceTests`). *Manual repro needs a planted `Path=/` cookie; record as covered by automated tests. If a tester hits a stuck `/refresh 401` loop in Firefox after a deploy, a single re-sign-in clears it.*

15. Traceability matrix (feature → cases → API)

| Feature area | Test cases | Key API endpoints |
|---------------------------|---|---|
| OAuth sign-in (Google/MS) | SMK-02, AUTH-02, DSK-02, AND-02 | GET <code>/api/auth/login/{provider}</code> , GET <code>/api/auth/callback/{provider}</code> , native login/callback/exchange |
| Magic link (web) | AUTH-01, 05, 06, MAIL-02 | POST <code>/api/auth/magic-link/send</code> , GET <code>/api/auth/magic-link/verify</code> |
| Email OTP | SMK-01/05/06, AUTH-03/04, DSK-01, AND-01, MAIL-01 | POST <code>/api/auth/otp/send</code> , POST <code>/api/auth/otp/verify</code> |

| Feature area | Test cases | Key API endpoints |
|--|--|--|
| Rate limiting / abuse guard | AUTH-11 | POST /api/auth/otp/send, .../magic-link/send, .../otp/verify (429) |
| Session / refresh / sign-out | SMK-03/04, DSK-03/06, AND-03, SEC-03, SEC-05 (legacy-cookie self-heal) | POST /api/auth/refresh, POST /api/auth/logout, GET /api/auth/me |
| Enumeration / error handling | AUTH-04/07/08/09 | (send + verify endpoints; /login query states) |
| Onboarding (auto tenant) | ONB-01/02, SMK-01 | (provisioned on first auth) |
| Household view/rename | HH-01/02 | GET /api/household, PUT /api/household |
| Members (remove/leave/transfer/dissolve) | HH-03..08 | DELETE /api/household/members/{id}, POST /api/household/leave, POST /api/household/transfer-ownership |
| Invitations | INV-01/06/07/08, MAIL-03 | POST /api/household/invitations, GET /api/household/invitations, POST .../{id}/regenerate, DELETE .../{id} |
| Join / accept | INV-02/03/04/05 | POST /api/household/invitations/accept |
| Linked accounts | SET-01..06, DSK-05 | GET /api/auth/logins, POST /api/auth/link/{provider}, DELETE /api/auth/logins/{provider} |
| Localization | I18N-01..04 | PUT /api/auth/locale (+ resx) |
| Emails / branding | MAIL-01..04, I18N-04 | (SMTP via Mailpit) |
| Tenant isolation / auth guards | SEC-01..05 | (all [Authorize] endpoints; write-stamping + reuse detection are automated) |
| Platform health / readiness | SMK-07 | GET /health, GET /health/ready |
| Transactional email delivery | (all email cases) | async via the outbox dispatcher (OutboxMessages) |
| Billing (API-only, no UI) | covered by Api.Tests (Billing*/Entitlement* tests); E2E pending | POST /api/billing/checkout, .../portal, .../webhook |
| Audit log (API-only) | covered by Api.Tests (AuditLogTests) | append-only IAuditLog + interceptor |

Per-client coverage: Web = full (all suites). Desktop = DSK-01..07 + shared-UI spot checks. Android = AND-01..06 + shared-UI spot checks. Magic link is **web-only** by design.

16. Sign-off sheet

Record one row per executed case. Build = API/web commit SHA (`git rev-parse --short HEAD`).

| Case ID | Client | Result (P/F/Blocked/N-A) | Tester | Build (SHA) | Date | Notes / defect link |
|-----------|--------|--------------------------|--------|-------------|------|---------------------|
| QA-SMK-01 | Web | | | | | |
| QA-SMK-02 | Web | | | | | |
| ... | | | | | | |

Release gate (suggested): all ● Smoke Smoke + all ● Core Core cases Pass on Web; ● Smoke Smoke Pass on Desktop and Android; no open Critical/High defects. ● Edge Edge cases triaged (Pass or accepted-known-issue).

17. Notes for maintainers

- This plan is the manual counterpart to the automated `tests/E2E.Tests` (Playwright/NUnit) suite. That suite now covers the OTP auth happy-path and guards (`AuthFlowTests` over `LoginPage`, reading codes from Mailpit via the `Mailpit` REST client) — see `tests/E2E.Tests/README.md` for the run procedure (it documents the

same Mailpit SMTP override noted in §1.1). The Gherkin blocks here are written to be lifted directly into new E2E scenarios — keep the two in sync as automation grows.

- When you add an app-specific domain feature on top of this template, add a matching suite here and a row in the traceability matrix (§15) so "entire functionality" stays honest.
- `docs/FEATURES.md` describes the same JWT-based flows at the design level; this plan is their step-by-step verification. Keep the two in sync when behavior changes.
- **Updated 2026-06-22** for the security/quality remediation: OTP lockout is now ****cumulative per email and resend-proof (QA-AUTH-04); passwordless send/verify are rate-limited (QA-AUTH-11); the Settings Unlink**** now requires a fail-closed confirmation (QA-SET-04); and server-side hardening (write-stamping, refresh-reuse detection, fail-closed takeover guard) is captured as automated coverage (QA-SEC-05).
- **Updated 2026-06-23:** QA-AUTH-02 now documents the **tenant-gated same-email auto-link** rule (Microsoft trusted only on the consumer's tenant; work/school + unknown providers fail closed — audit MITI-3); and QA-SEC-05 adds the **legacy refresh-cookie self-heal** as automated coverage (`CookieServiceTests`).
- **Updated 2026-06-26** for the platform-foundation work (JOBS/BILLING/OBS, ADR-006/007/008):
- **Transactional email is now delivered asynchronously** via the outbox dispatcher — a few-second delay, and the send request always succeeds (reliability moved to the background). Affects every email-based case; see the §1.1 and §11 notes. The E2E `AuthFlowTests` (which reads OTP codes from Mailpit) now traverses this async path — confirm it waits/polls for the email rather than assuming instant delivery.
- Added **API health/readiness** endpoints + smoke case **QA-SMK-07** (`/health`, `/health/ready`).
- The **billing API** (`checkout/portal/webhook`), the **append-only audit log**, and **OpenTelemetry** telemetry are API/operational-level with **no client UI** — covered by `tests/Api.Tests`, E2E pending; manual cases will follow when UI exists (see §2 + §15).